

Bundle File Tool v2.1 Build 100

Tri-UI Feature and System Specification

Prepared by Paul, Lead Analyst - Proposed for Ringo, George, and John - 2026-07-08

1. Executive Conclusion

The Bundle File Tool should move to v2.1 Build 100 as a clean architectural target with one shared, headless operation layer and three complete user interfaces: CLI, Tkinter GUI, and local Web UI. The current Build 101 source already contains valuable core modules (`models`, `parser`, `writer`, `validators`, `config`, `logging`, `profiles`) and a dual-mode Tkinter GUI, but it is not yet a true three-interface product because the Web UI is absent and the CLI/Tkinter flows still call lower-level core objects directly.

The central recommendation is to add a Core Service Facade inside `src/core/` and require every UI to call only that facade for application operations. The parser, writer, validator, profile registry, configuration manager, and logger remain in `core/`, but the public application contract becomes `BundleToolService`. This gives the team a single implementation of bundle, unbundle, validate, preview, configuration, safety, and profile behavior that can be exercised identically by CLI, Tkinter, and Web.

This is the right target because the existing v2.1 specification already requires symmetrical bundle/unbundle workflows, profile extensibility, configuration-driven behavior, zero-regression protection, and safety by default. The alternatives analysis also supports the strategic value of a tri-UI product because the competitive market is strong at one-way repo-to-text ingestion but comparatively weak at native bidirectional bundle/unbundle workflows with safe extraction.

2. Mission Statement

Deliver Bundle File Tool v2.1 Build 100 as a professional, internally governed developer utility that can:

1. Bundle a file tree into a deterministic text artifact.
2. Unbundle that artifact back to a safe output folder.
3. Validate bundle structure and safety before write operations.
4. Support multiple bundle profiles using a deterministic registry.
5. Provide identical operations through CLI, Tkinter GUI, and local Web UI.
6. Package and stage releases through the new `PREP\_AND\_STAGE\_BFT.bat` delivery process.

3. Source Basis and Evidence Summary

The v2.1 master specification establishes the intended architecture: symmetrical workflows, profile-based extensibility, configuration-centric behavior, zero-regression against v1.1.5 behavior, and safety mechanisms such as path validation, overwrite policy, and file size limits. The same specification describes `core/models.py`, `core/parser.py`, `core/writer.py`, `core/profiles/\*`, `core/validators.py`, and `ConfigManager` as the intended component responsibilities.

The current source bundle shows a valuable but incomplete implementation. It includes `cli.py`, `main.py`, `core/` modules, `ui/` Tkinter frames, and generated ID files. It does not include a `src/web/` package, web server entry point, static web assets, or API endpoints. It also shows that `MarkdownFenceProfile` exists as a module, but the current `ProfileRegistry` registers only `PlainMarkerProfile`; meanwhile configuration and documentation expect `md\_fence` and `jsonl` to be valid profiles. This creates a critical gap between configured defaults, specification, and runtime behavior.

The `PREP\_AND\_STAGE\_BFT.bat` script defines the new delivery workflow: preflight project root, read `VERSION.txt`, create a snapshot, archive the snapshot, delete older snapshot folders only after archive verification, stage exactly one delivery zip from Downloads, verify an installer and `\_bundletool\_incoming`, run the installer unless `/norun`, and increment the build after successful install.

4. Current Build 101 Baseline Assessment

4.1 Existing strengths

Area	Observed strength	Keep for Build 100
Data model	`BundleEntry` and `BundleManifest` are established and carry path/content/metadata concepts.	Yes. Make them the stable domain objects under the service facade.
Parser/writer split	`BundleParser`, profile classes, `BundleWriter`, and `BundleCreator` exist.	Yes. Preserve the split and expose through service operations.
Safety	Path validation, glob filtering, file size validation, overwrite policy, dry run, and nested bundle detection exist in some form.	Yes. Make safety reports first-class service outputs.
CLI	`bundle`, `unbundle`, and `validate` commands exist.	Yes. Refactor handlers to call service methods only.
Tkinter GUI	Dual-mode GUI with `UnbundleFrame`, `BundleFrame`, `MainWindow`, and `ModeManager` exists.	Yes. Preserve visual behavior while moving business calls behind the service facade.
Delivery direction	Project already has headers, lifecycle status, and team governance artifacts.	Yes. Normalize and enforce in acceptance tests.

4.2 Blocking gaps and inconsistencies

Gap	Finding	Build 100 requirement
No Web UI	Source bundle has no `src/web/`, Flask/FastAPI app, HTML/CSS/JS, or local web endpoints.	Add complete local Web UI with stable endpoints and DOM IDs.
Core is not fully isolated from UI orchestration	Tkinter frames instantiate `BundleParser`, `BundleWriter`, `BundleCreator`, and `ProfileRegistry` directly.	UI layers must call `BundleToolService`; direct parser/writer calls move out of UI.
Profile registry incomplete	`MarkdownFenceProfile` exists, but registry currently registers only `PlainMarkerProfile`; `jsonl` is configured/expected but not implemented.	Registry must register `plain_marker`, `md_fence`, and `jsonl` in deterministic order.
Version drift	File headers reference mixed versions such as 2.1.0, 2.1.1, and 2.1.11 while user baseline is V2.0 Build 101.	Add single `src/core/version.py` and `VERSION.txt=2.1.100`; headers should stop being runtime truth.
Test compatibility shims in production path	`writer.py` contains monkeypatch behavior for `builtins.all` and proxy length objects to satisfy tests.	Remove production monkeypatches; fix tests or isolate compatibility helpers under tests only.
Delivery standard not yet integrated into package production	PREP script expects a precise install zip and incoming folder pattern, but no package creation contract is specified in source.	Add release packaging script and installer contract that generates `INSTALL_BUNDLETOL_*.zip`.
Config/schema vs runtime mismatch	Config validates `jsonl` but no JSONL profile implementation is present.	Implement JSONL or remove it from allowed values; Build 100 should implement it.

5. Build 100 Feature Definition

5.1 Feature ID

BFT-2.1-B100-TRI-UI-HEADLESS-CORE

5.2 Feature title

Tri-UI Bundle File Tool with Shared Headless Core and Team Delivery Standard

5.3 Feature scope

Build 100 adds or formalizes:

1. A public core service facade for all operational workflows.
2. Complete parity across CLI, Tkinter GUI, and local Web UI.
3. Deterministic profile registry with `plain\_marker`, `md\_fence`, and `jsonl`.
4. Web API and browser UI backed by the same core service.
5. Delivery package conventions compatible with `PREP\_AND\_STAGE\_BFT.bat`.
6. QA/QC acceptance gates for operation parity and release staging.

5.4 Explicit non-goals for Build 100

Build 100 should not add Tree-sitter semantic compression, MCP server mode, cloud processing, or multi-user authentication. These are strong future features, but Build 100 must first finish the architecture split and tri-UI parity without destabilizing the existing bundle/unbundle function.

6. Target Architecture

6.1 Required dependency direction

```
CLI UI -> core.service -> core.parser/core.writer/core.validators/core.config/core.logging
Tkinter GUI -> core.service -> core.parser/core.writer/core.validators/core.config/core.logging
Web UI/API -> core.service -> core.parser/core.writer/core.validators/core.config/core.logging

core/* MUST NOT import tkinter, flask, argparse, browser assets, or UI modules.
ui/* MUST NOT call BundleParser, BundleWriter, BundleCreator, or ProfileRegistry directly.
web/* MUST NOT duplicate parser/writer logic.
cli.py MUST NOT duplicate parser/writer logic.
```

6.2 Proposed repository layout

```
bundle_file_tool_v2/
VERSION.txt # 2.1.100 at target freeze
bundle_config.json
pyproject.toml
requirements.txt
PREP_AND_STAGE_BFT.bat
src/
cli.py # CLI adapter only
main.py # Tkinter launcher only
web_main.py # Web launcher only
core/
__init__.py
version.py # single source of runtime version truth
service.py # public operation facade
operations.py # request/result dataclasses, if kept separate
models.py
parser.py
writer.py
validators.py
config.py
```

```

logging.py
bundle_integrity.py
profiles/
__init__.py
base.py
plain_marker.py
markdown_fence.py
jsonl.py
ui/
__init__.py
main_window.py
mode_manager.py
unbundle_frame.py
bundle_frame.py
settings_dialog.py
web/
__init__.py
app.py
routes.py
static/
index.html
app.js
styles.css
templates/
index.html # optional if not using pure static
tests/
unit/
integration/
coverage_extra/
ui_contract/
web_contract/
delivery/
docs/
BFT_v2_1_Build100_Tri_UI_Feature_and_System_Spec.pdf
sample/
sample_project_plain_marker.txt
sample_project_markdown_fence.txt
sample_project_jsonl.jsonl
scripts/
build_bft_delivery.py
run_qc_bft.py

```

7. Core Service Facade Specification

7.1 Required module

`src/core/service.py`

7.2 Required public class

`BundleToolService`

7.3 Required service methods

Method	Purpose	Used by
`get_version()`	Return canonical app/version/build metadata.	CLI, GUI, Web
`get_profiles()`	Return available profile names, display names, capabilities, and default profile.	CLI, GUI, Web
`load_config()`	Return normalized effective configuration.	CLI, GUI, Web
`save_config(config_patch)`	Validate and persist allowed config changes.	GUI, Web, optional CLI
`discover_files(request)`	Scan source paths with allow/deny globs and size policy.	CLI, GUI, Web

Method	Purpose	Used by
`preview_bundle(request)`	Return manifest summary and bundle text preview without writing.	GUI, Web, optional CLI
`create_bundle(request)`	Create bundle text and optionally write to file.	CLI, GUI, Web
`parse_bundle(request)`	Parse text/file/clipboard input into manifest summary.	CLI, GUI, Web
`validate_bundle(request)`	Return validation report without extraction.	CLI, GUI, Web
`plan_extract(request)`	Dry-run extraction plan with paths, conflicts, renames, warnings.	GUI, Web, CLI dry-run
`extract_bundle(request)`	Apply extraction according to overwrite policy and safety validation.	CLI, GUI, Web
`export_logs(request)`	Return or write session log summary.	GUI, Web, CLI

7.4 Service result rules

All service methods must return structured result objects and avoid UI-specific side effects. A service result must contain:

```

success: bool
operation: str
profile: str | None
summary: dict
warnings: list[str]
errors: list[str]
files: list[dict]
output_text: str | None
output_path: str | None
log_file: str | None

```

Exceptions are still allowed for programmer errors, but expected user errors such as invalid profile, blocked path, missing source path, denied glob, file too large, and parse failure should be returned as service errors so every UI can render them consistently.

8. UI Parity Requirements

8.1 Shared operation parity

Operation	CLI	Tkinter GUI	Web UI
Discover source files	Must	Must	Must
Select/refine file list	Include/exclude flags	Tree selection	Browser tree/checklist
Preview bundle text	Optional command/flag	Must	Must
Create bundle to file	Must	Must	Must
Copy bundle to clipboard	Not required	Must	Browser copy button
Parse/open bundle	Must	Must	Must
Validate bundle	Must	Must	Must
Dry-run extraction	Must	Must	Must
Extract bundle	Must	Must	Must
Configure profile and safety	CLI flags/config	Settings dialog	Settings page
View operation errors	stderr/JSON option	message/log panel	alert/log panel

8.2 CLI requirements

The CLI must be an adapter over `BundleToolService`, preserving existing commands and adding parity flags where needed.

```
python src\cli.py profiles
python src\cli.py validate <bundle_file> [--profile <name>] [--json]
python src\cli.py bundle <source_paths...> --output <file> [--profile <name>] [--base-path <dir>] [--include <glob>]
[--exclude <glob>] [--dry-run] [--json]
python src\cli.py unbundle <input_file> --output <dir> [--profile <name>] [--overwrite prompt|skip|rename|overwrite]
[--dry-run] [--no-headers] [--json]
python src\cli.py web [--host 127.0.0.1] [--port 8765] [--no-browser]
```

Acceptance rule: CLI handlers must not construct `BundleParser`, `BundleWriter`, `BundleCreator`, or `ProfileRegistry` directly.

8.3 Tkinter GUI requirements

The existing Tkinter GUI should be preserved visually where possible, but business operations move behind `BundleToolService`.

Required GUI elements:

Element ID/name	Requirement
Mode switcher	Preserve Un-bundle / Bundle modes.
Un-bundle frame	Open bundle, validate, dry run, extract, output directory, overwrite policy, add headers.
Bundle frame	Select source, tree selection, profile selector, include/exclude controls, live preview, create/copy bundle.
Settings dialog	General, Bundle, Safety, Web tabs.
Log/status panel	Consistent service result display.
Progress indicator	At minimum indeterminate progress for long scans/writes.

Acceptance rule: `ui/unbundle\_frame.py` and `ui/bundle\_frame.py` may import `BundleToolService` and DTOs, but must not import `core.parser`, `core.writer`, or `core.profiles` directly.

8.4 Web UI requirements

Build 100 must add a local browser UI. The recommended implementation is Flask because it is lightweight and familiar for local Python desktop tooling. FastAPI remains acceptable if John and George prefer it, but the dependency and launch command must be locked before implementation begins.

Required files:

```
src/web/app.py
src/web/routes.py
src/web/static/index.html
src/web/static/app.js
src/web/static/styles.css
src/web_main.py
```

Required endpoints:

Endpoint	Method	Purpose
`/`	GET	Serve Web UI.
`/api/health`	GET	Return app status and version.
`/api/version`	GET	Return `2.1.100` metadata.
`/api/profiles`	GET	Return profile list/capabilities.
`/api/config`	GET	Return effective config.
`/api/config`	PUT	Save validated config patch.
`/api/files/discover`	POST	Discover source files.
`/api/bundle/preview`	POST	Return bundle preview text and summary.

Endpoint	Method	Purpose
`/api/bundle/create`	POST	Write bundle or return text.
`/api/unbundle/validate`	POST	Validate bundle text/file path.
`/api/unbundle/plan`	POST	Return dry-run extraction plan.
`/api/unbundle/extract`	POST	Extract with overwrite policy.
`/api/logs/session`	GET	Return current session log summary.

Required stable DOM IDs:

```
tabBundle
btnTabBundle
panelBundle
inputSourcePath
btnDiscoverFiles
fileSelectionTree
selectProfile
textareaBundlePreview
btnCreateBundle
btnCopyBundle

tabUnbundle
btnTabUnbundle
panelUnbundle
inputBundlePath
textareaBundleText
btnValidateBundle
inputOutputDir
selectOverwritePolicy
checkboxDryRun
btnExtractBundle

panelSettings
panelStatus
operationLog
```

Required global JavaScript function:

```
function showTab(tabName, buttonEl) { ... }
```

Per team directive, this function name must remain stable once introduced.

9. Profile Requirements

9.1 Required profiles for Build 100

Profile	Build 100 status	Requirement
`plain_marker`	Existing but must be hardened	Preserve v1/v2 compatibility; header-context boundary detection required.
`md_fence`	Module exists but not fully wired	Register in deterministic order; verify round trip with canonical sample.
`jsonl`	Missing implementation	Implement as machine-readable profile or remove from config. Recommendation: implement.

9.2 Registry order

The registry order must be deterministic:

```
PlainMarker -> MarkdownFence -> JSONL
```

Auto-detection must use profile `detect_format()` methods in that order unless a user explicitly supplies `--profile` or equivalent UI selection.

9.3 JSONL profile minimal schema

Each line is one JSON object:

```
{ "file": "src/example.py", "encoding": "utf-8", "eol": "LF", "mode": "text", "content": "print('hello')\n", "checksum": "option al-sha256", "file_size_bytes": 23 }
```

Required keys: `file`, `encoding`, `eol`, `mode`, `content`.

Optional keys: `checksum`, `file_size_bytes`, `content_type`.

10. Configuration Schema Additions

Build 100 should retain the v2.1 nested schema and add Web/delivery support.

```
{
  "global_settings": {
    "input_dir": "",
    "output_dir": "",
    "log_dir": "logs",
    "relative_base_path": "",
    "last_source_dir": "",
    "last_bundle_save_dir": ""
  },
  "session": {
    "first_launch": false,
    "window_geometry": "",
    "last_ui": "gui"
  },
  "app_defaults": {
    "default_mode": "unbundle",
    "bundle_profile": "md_fence",
    "add_headers": true,
    "encoding": "auto",
    "eol": "auto",
    "overwrite_policy": "prompt",
    "dry_run_default": true,
    "treat_binary_as_base64": true
  },
  "web": {
    "host": "127.0.0.1",
    "port": 8765,
    "auto_open_browser": true,
    "enable_write_operations": true,
    "require_extract_confirmation": true,
    "max_request_mb": 25
  },
  "safety": {
    "allow_globs": ["**/*.py", "**/*.md", "**/*.json", "**/*.toml", "**/*.html", "**/*.css", "**/*.js"],
    "deny_globs": ["**/.venv/**", "**/__pycache__/**", "**/.pytest_cache/**", "**/dist/**", "**/build/**",
    "**/_bundle*.txt", "**/*.zip", "**/*.tar", "**/*.tar.*"],
    "max_file_mb": 10
  }
}
```

11. Delivery Standard Integration

11.1 Version rule

The target release line is:

```
VERSION.txt = 2.1.100
```

When `PREP\_AND\_STAGE\_BFT.bat` successfully runs the installer, the script increments the build. Therefore, a successful Build 100 delivery should leave the installed working tree at:

```
VERSION.txt = 2.1.101
```

This is expected behavior and should not be treated as version drift.

11.2 Delivery package contract

The package builder must generate exactly one zip matching:

```
%USERPROFILE%\Downloads\INSTALL_BUNDLETOOL_*.zip
```

The zip must extract into the project root and contain:

```
INSTALL_BUNDLETOOL_<version>_Build_<build>.bat
_bundletool_incoming\
src\...
tests\...
docs\...
config\... # if present
requirements.txt
pyproject.toml
VERSION.txt
CHANGELOG_BFT_<version>_<build>.csv
```

The installer must copy from `\_bundletool\_incoming` into the live project tree, preserve backups as directed by the staging flow, and exit non-zero on any failed file operation.

11.3 Required pre-delivery evidence

Every delivery should include:

1. Pytest result summary.
2. Code catalog comparison report.
3. Changelog CSV.
4. Updated PDF documentation.
5. Delivery zip name and hash.
6. Confirmation that `PREP\_AND\_STAGE\_BFT.bat /dryrun` passes before live staging.

12. Implementation Plan

Phase 0 - Baseline normalization

Owner: John, with George review and Paul QA.

1. Add `VERSION.txt` with `2.1.100` for the target branch.

2. Add ``src/core/version.py`` and update UI/version display to use it.
3. Confirm source files under current Build 101 and update manifest/catalog.
4. Remove or quarantine production test shims from ``writer.py``.
5. Confirm no source headers use ``# FILE:`` as a source header standard.

Exit gate: Project launches existing CLI and GUI exactly as before, with no feature regression.

Phase 1 - Core service facade

Owner: John; architecture review by George.

1. Add ``src/core/service.py``.
2. Add request/result dataclasses in ``src/core/operations.py`` or inside ``service.py``.
3. Move CLI/Tkinter orchestration logic into service methods.
4. Keep lower-level parser/writer APIs intact to avoid regression.

Exit gate: Unit tests can perform bundle, unbundle, validate, discover, preview, and extract through ``BundleToolService`` without importing any UI module.

Phase 2 - Profile registry completion

Owner: John; Paul validates with round-trip tests.

1. Register ``PlainMarkerProfile``, ``MarkdownFenceProfile``, and ``JsonlProfile``.
2. Implement ``src/core/profiles/jsonl.py``.
3. Create canonical samples for all three formats.
4. Add round-trip tests for each profile.

Exit gate: ``get_profiles()`` returns all three profiles and all canonical samples validate.

Phase 3 - CLI adapter refactor

Owner: John.

1. Rewrite CLI handlers to call ``BundleToolService`` only.
2. Add ``profiles`` command and ``--json`` output where useful.
3. Preserve existing command syntax.
4. Add service-level structured errors to stderr and exit codes.

Exit gate: CLI tests pass and prove no direct parser/writer/profile construction in ``cli.py``.

Phase 4 - Tkinter adapter refactor

Owner: John; UX review by Ringo and Paul.

1. Update ``UnbundleFrame`` to use service methods.

2. Update `BundleFrame` to use service methods.
3. Add/finish Settings dialog tabs, including Web settings.
4. Ensure UI behavior remains consistent with v2.1 specification.

Exit gate: Manual GUI smoke test plus integration tests pass for both modes.

Phase 5 - Web UI

Owner: John; architecture review by George; QA by Paul.

1. Add local web package and launcher.
2. Implement stable endpoints listed in this spec.
3. Add browser UI with bundle and unbundle tabs.
4. Add DOM IDs and global `showTab(tabName, buttonEl)`.
5. Add web endpoint tests using Flask/FastAPI test client.

Exit gate: Web UI can discover files, preview bundle, create bundle, validate bundle, dry-run extract, and extract via the same core service.

Phase 6 - Delivery tooling

Owner: John; process verification by Paul.

1. Add `scripts/build\_bft\_delivery.py` or equivalent batch wrapper.
2. Generate `INSTALL\_BUNDLETOOL\_\*.zip` with required contents.
3. Verify `PREP\_AND\_STAGE\_BFT.bat /dryrun` assumptions.
4. Document exact release process in `docs/`.

Exit gate: A dry-run staging workflow resolves all paths and package contents without changes.

Phase 7 - QA/QC freeze

Owner: Paul with George review.

1. Run full pytest.
2. Run code catalog and comparison tool.
3. Verify UI/API mapping table.
4. Verify release artifact hash.
5. Confirm no direct UI-to-parser/writer imports remain.

Exit gate: Build 100 can be approved for team installation.

13. Acceptance Test Matrix

Test ID	Category	Scenario	Expected result
BFT-B100-001	Service	<code>`BundleToolService.get_profiles()`</code>	Returns <code>`plain_marker`</code> , <code>`md_fence`</code> , <code>`jsonl`</code> in deterministic order.
BFT-B100-002	Service	Bundle canonical sample project with each profile.	Bundle text generated; result success true.
BFT-B100-003	Service	Round-trip each canonical sample.	Restored files match expected content, encoding, and EOL policy.
BFT-B100-004	Safety	Attempt extraction path <code>`../../bad.txt`</code> .	Operation blocked; no file written outside output root.
BFT-B100-005	Safety	Bundle source containing nested historical bundle.	Operation fails loud before writing output.
BFT-B100-006	CLI	<code>`python src/cli.py profiles --json`</code> .	JSON contains all profiles and capabilities.
BFT-B100-007	CLI	Existing <code>`bundle`</code> , <code>`unbundle`</code> , <code>`validate`</code> commands.	Existing syntax still works.
BFT-B100-008	GUI	Open bundle, validate, dry-run extract.	UI shows service result and writes nothing in dry run.
BFT-B100-009	GUI	Select source, preview, create bundle.	Preview and output match CLI for same request.
BFT-B100-010	Web	<code>`/api/health`</code> .	HTTP 200 with version and status.
BFT-B100-011	Web	Discover, preview, create bundle.	Response matches service and CLI output.
BFT-B100-012	Web	Validate and dry-run extraction.	Same result as CLI/GUI.
BFT-B100-013	Delivery	<code>`PREP_AND_STAGE_BFT.bat /dryrun`</code> .	Shows plan and changes nothing.
BFT-B100-014	Delivery	Delivery zip contents.	Exactly one installer and <code>`_bundletool_incoming`</code> present.
BFT-B100-015	Governance	Code catalog comparison.	No unreviewed deletions, signature drift, or orphaned public endpoints.

14. Team Communication and Assignments

To George - Architecture review items

Please validate the service facade boundary and the dependency rule that all UIs call ``BundleToolService`` only. The key architectural decision is whether Web uses Flask or FastAPI. My recommendation is Flask for Build 100 because it keeps the local app lightweight and easier to package; FastAPI is acceptable if the dependency contract and test client are locked before implementation.

To John - Development priorities

The highest priority is not the Web UI itself; it is the service facade that makes the Web UI safe to add. Implement ``BundleToolService``, then refactor CLI, then refactor Tkinter, then add Web. Also fix profile registration before UI work, because the current configuration/default profile direction is inconsistent unless ``md_fence`` and ``jsonl`` are wired and tested.

To Ringo - Product/governance decision

Build 100 should be treated as a stabilization and architecture-completion release, not a feature expansion release. The user-visible headline is "three complete UIs," but the product value comes from having the same safe core behavior under all three. I recommend approving Build 100 with the service facade, Web UI, profile completion, delivery standard, and parity tests as the mandatory scope.

15. Final Recommendations

1. Approve ****BFT-2.1-B100-TRI-UI-HEADLESS-CORE**** as the Build 100 feature target.
2. Freeze target version at `2.1.100` before packaging; accept post-install increment to `2.1.101` as the delivery script's intended result.
3. Require a new `BundleToolService` facade before Web work starts.
4. Require all three profiles (`plain\_marker`, `md\_fence`, `jsonl`) to register and pass round-trip tests.
5. Remove production test shims from core runtime paths.
6. Add local Web UI using stable endpoints and stable DOM IDs.
7. Do not ship until CLI, Tkinter, and Web demonstrate identical results for the same service requests.
8. Use `PREP\_AND\_STAGE\_BFT.bat` as the official staging process and make the package builder produce exactly the artifact that script expects.

Appendix A - Proposed Web API JSON examples

Health response

```
{
  "success": true,
  "app": "Bundle File Tool",
  "version": "2.1.100",
  "status": "ok"
}
```

Bundle preview request

```
{
  "source_paths": ["C:/Users/mpw/Python/bundle_file_tool_v2/src"],
  "base_path": "C:/Users/mpw/Python/bundle_file_tool_v2",
  "profile": "md_fence",
  "include": ["src/**/*.py"],
  "exclude": ["**/__pycache__/**"],
  "dry_run": true
}
```

Service result shape

```
{
  "success": true,
  "operation": "bundle.preview",
  "profile": "md_fence",
  "summary": {
    "file_count": 21,
    "text_count": 21,
    "binary_count": 0,
    "total_size_bytes": 288593
  },
  "warnings": [],
  "errors": [],
  "files": [],
  "output_text": "...bundle preview text...",
  "output_path": null,
  "log_file": "logs/session.jsonl"
}
```

Appendix B - No-code-change changelog for this analysis deliverable

project,date,change_type,description,files_changed

Bundle File Tool v2,2026-07-08,analysis/specification,"Created proposed v2.1 Build 100 Tri-UI headless-core feature and system specification.", "No repository code files changed"